

Gesture Recognition

Gesture Recognition

1. Course Content

Function Introduction:

Mediapipe Hands

2. Program Startup

3. Core Code Analysis

1. Course Content

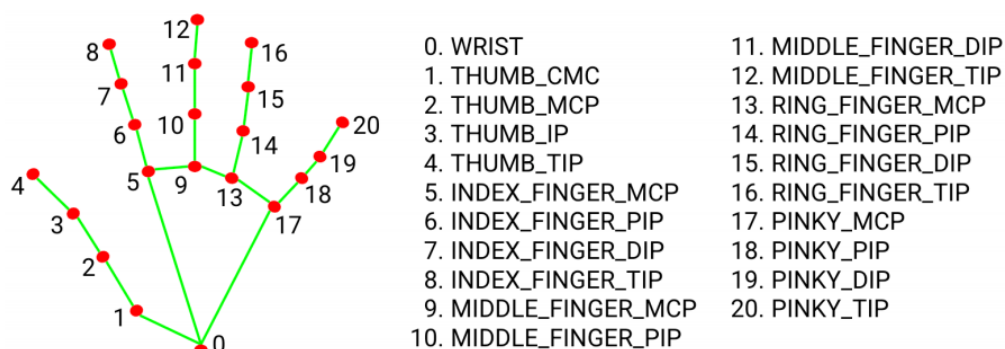
This course explains how to subscribe to image topics to obtain images and use mediapipe for gesture recognition

Function Introduction:

MediaPipe is a data stream processing machine learning application development framework developed and open-sourced by Google. It is a graph-based data processing pipeline used to build applications that use various forms of data sources, such as video, audio, sensor data, and any time series data. MediaPipe is cross-platform and can run on embedded platforms (such as Raspberry Pi), mobile devices (iOS and Android), workstations and servers, and supports mobile GPU acceleration. MediaPipe provides cross-platform, customizable ML solutions for real-time and streaming media. MediaPipe's core framework is implemented in C++ and provides support for languages such as Java and Objective C. MediaPipe's main concepts include Packet, Stream, Calculator, Graph, and Subgraph.

Mediapipe Hands

MediaPipe Hands is a high-fidelity hand and finger tracking solution. It uses machine learning (ML) to infer 21 3D hand coordinates from a single frame. After palm detection on the entire image, the hand landmark model performs precise keypoint positioning through regression for 21 3D hand joint coordinates within the detected hand region, i.e., direct coordinate prediction. The model learns consistent internal hand posture representations and is robust even to partially visible hands and self-occlusion. To obtain ground truth data, approximately 30K real-world images were manually annotated with 21 3D coordinates. As shown below, to better cover possible hand poses and provide additional supervision on the nature of hand geometry, high-quality synthetic hand models were also drawn in various backgrounds and mapped to corresponding 3D coordinates.



Therefore, as long as we can obtain the coordinate values of each joint, we can perform gesture recognition through calculation.

2. Program Startup

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

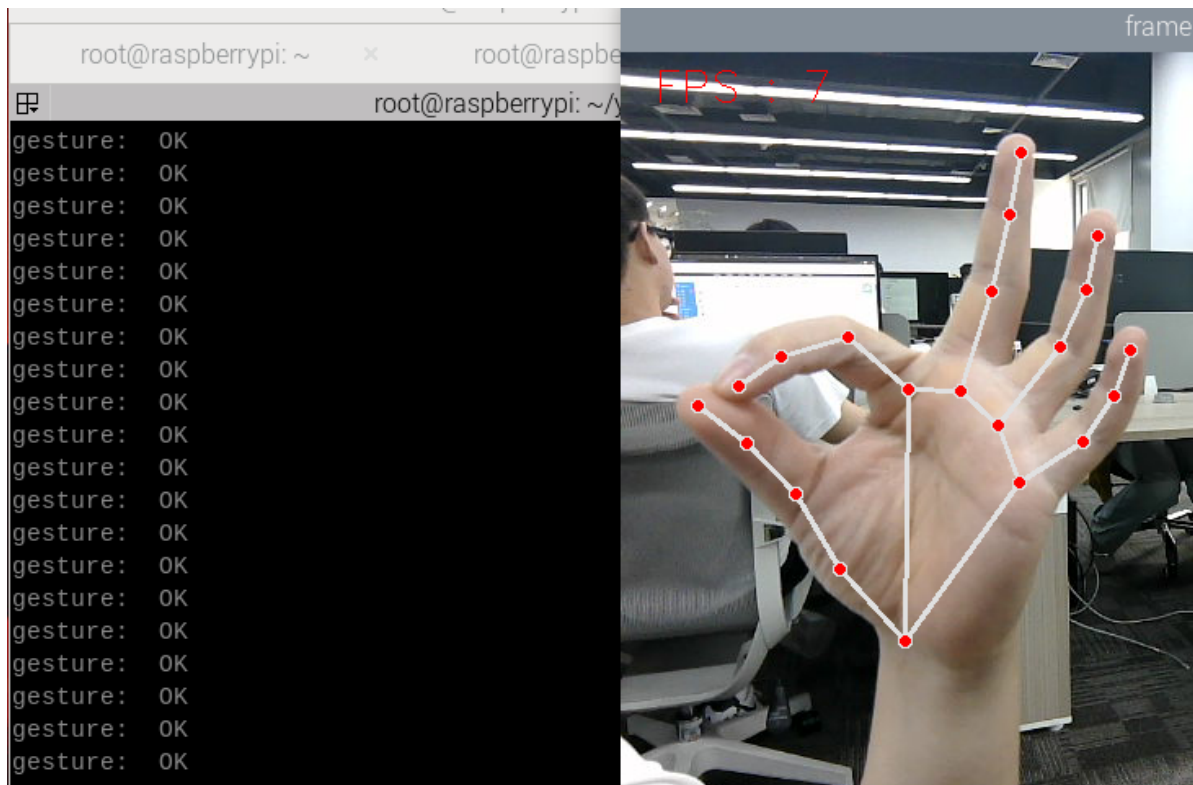
Open a terminal inside the container,

```
exec_m3
```

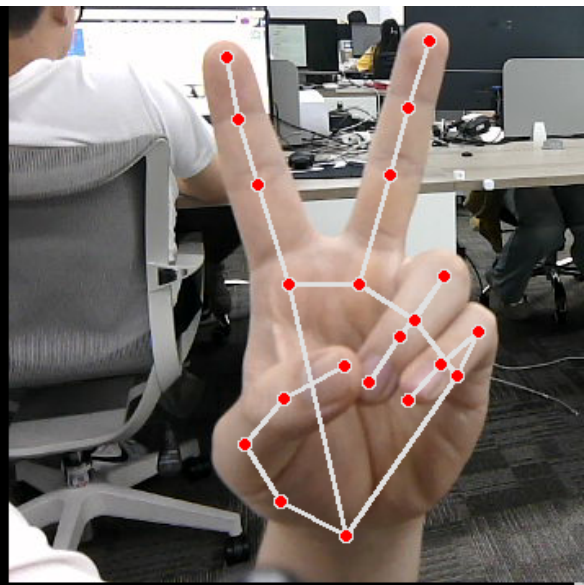
- Run gesture detection example program

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=mediapipe_gesture
```

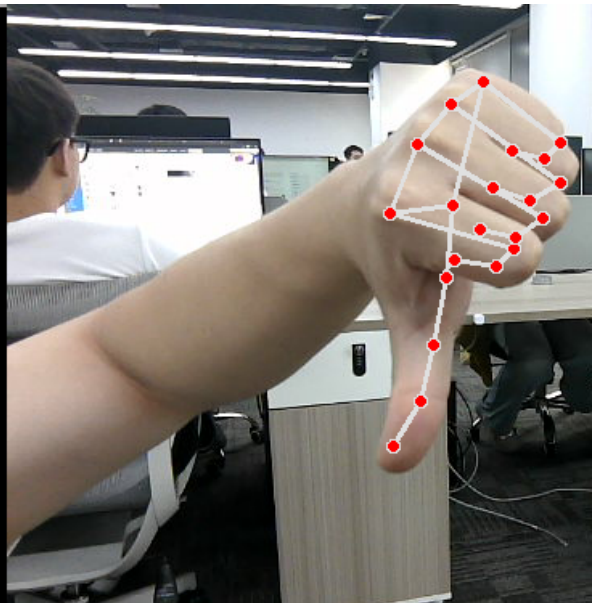
After the program starts, as shown in the figure below, it can recognize three gestures: 【OK】 , 【Yes】 , and 【Thumb_down】 ,



```
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
gesture: Yes
```



```
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
gesture: Thumb_down
```



3. Core Code Analysis

Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/mediapipe_gesture.py
```

Import necessary library files,

```
import cv2
import os
from sensor_msgs.msg import Image
import message_filters
from cv_bridge import CvBridge
import cv2 as cv
from arm_msgs.msg import ArmJoints
import time
from M3Pro_demo.media_library import *
from rclpy.node import Node
import rclpy
from message_filters import Subscriber,
Timesynchronizer, ApproximateTimesynchronizer
```

```
from sensor_msgs.msg import Image
import threading
```

Initialize some variables and create subscribers and publishers,

```
def __init__(self, name):
    super().__init__(name)
    self.init_joints = [90, 150, 12, 20, 90, 0]
    self.rgb_bridge = CvBridge()
    #Create Medipipe recognition object
    self.hand_detector = HandDetector()
    self.pr_time = time.time()
    self.pTime = self.cTime = 0
    #Create color image topic subscriber
    self.rgb_image_sub = Subscriber(self, Image, '/camera/color/image_raw')
    #Publish initial recognition pose
    self.ts = ApproximateTimeSynchronizer([self.rgb_image_sub], 1, 0.5)
    self.ts.registerCallback(self.callback)
    time.sleep(2)
    self.start_time = 0.0
```

Color image callback function and image processing,

```
def callback(self,color_msg):
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_msg, "bgr8")
    self.process(rgb_image)

def process(self, frame):
    #Call the findHands function in the Medipipe recognition class
    frame, lmList, bbox = self.hand_detector.findHands(frame)
    #Check the length of lmList, if greater than 0, it means a hand was found
    if len(lmList) != 0:
        #Enter the thread processing gesture recognition function, the parameter
        #passed is lmList
        gesture = threading.Thread(target=self.Gesture_Detect_threading, args=
(lmList,bbox))
        gesture.start()
        gesture.join()
        self.cTime = time.time()
        fps = 1 / (self.cTime - self.pTime)
        self.pTime = self.cTime
        text = "FPS : " + str(int(fps))
        cv.putText(frame, text, (20, 30), cv.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0,
255), 1)
        if cv.waitKey(1) & 0xFF == ord('q'):
            cv.destroyAllWindows()
            cv.imshow('frame', frame)

def Gesture_Detect_threading(self, lmList,bbox):
    #Call the get_gesture function in the Medipipe recognition class
    gesture = self.hand_detector.get_gesture(lmList)
    print("gesture: ",gesture)
```

Regarding the definition of the `Medipipe` recognition class, it can be found in the `media_library` library. The location of this library is in the `m3_common_demo` package, with the path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/media_library.py
```

In this library, we use the native mediapipe library to extend and define many classes. Each class defines different functions. When needed, we can pass parameters into them. For example, in the HandDetector class, we define the following functions:

- findHands: Find hands
- fingersUp: Fingers extended downward
- ThumbTOforefinger: Detect the angle between thumb and index finger
- get_gesture: Detect gestures